

Este documento apresenta uma visão consolidada de todos os recursos, infraestrutura, modelagem de dados e rotas de API que foram efetivamente implementados no projeto LinguoUp até o momento.

1. Arquitetura e Configuração Global (Monorepo)

O projeto está estruturado como um **Monorepo** utilizando o gerenciador de pacotes `pnpm` (com workspaces definidos no `pnpm-workspace.yaml`). Essa estrutura separa as responsabilidades de forma clara e isolada:

- **apps/api**: Serviço backend desenvolvido em **NestJS** e TypeScript, contendo a lógica de negócios, banco de dados e APIs REST.
 - **apps/mobile**: Aplicação mobile-first desenvolvida em **React Native + Expo** e NativeWind (atualmente em estado de template/setup inicial).
 - **apps/web**: Painel administrativo web desenvolvido em **Next.js** App Router e Tailwind CSS (atualmente em estado de template/setup inicial).
 - **apps/frontend**: Configuração e base para testes E2E utilizando **Playwright**.
 - **packages/database**: Pacote compartilhado contendo as migrações, sementes (seeds) e a configuração do cliente **Prisma ORM**.
 - **packages/ui**: Placeholder de biblioteca de componentes visuais compartilhados.
 - **packages/config**: Arquivos de regras globais compartilhadas (ESLint, Prettier, TSConfig).
-

2. Banco de Dados & Modelagem (Prisma ORM)

O banco de dados configurado é o **PostgreSQL**, com o schema totalmente mapeado no **schema.prisma**. A estrutura implementada inclui:

Modelo	Descrição	Principais Campos
--------	-----------	-------------------

Entity	Description	Attributes
User	Registro central do usuário.	id, email, name, passwordHash, role (USER, ADMIN, SUPER_ADMIN), tenant_id
UserPreferences	Preferências de estudo do usuário.	targetLanguage, learningGoal, dailyGoalMinutes, preferredStudyTime
Lesson	Lições de idiomas cadastradas.	title, description, level, theme, durationMinutes, content (JSON)
LessonCompletion	Histórico de lições finalizadas.	userId, lessonId, completedAt, score, xpEarned
UserProgress	Acompanhamento do progresso geral.	totalXP, currentLevel, currentStreakDays, longestStreak, lastActivityDate
Achievement / UserAchievement	Sistema de conquistas e gamificação.	name, description, xpReward, criteria (JSON)
SpacedReviewItem	Itens do algoritmo de repetição espaçada (SM-2).	nextReviewAt, easeFactor, interval, repetitions
Notification	Alertas e lembretes gerados para o	type, message, readAt, createdAt

3. API de Autenticação e Segurança (NestJS)

O módulo de autenticação (AuthModule) está totalmente implementado e validado no backend NestJS (apps/api). Os recursos incluem:

Rotas e Endpoints (Base URL: /api/v1/auth)

- **POST /register:** Criação de novas contas de usuário.
- Valida a senha, executa hash criptográfico (bcrypt/argon2 equivalente) e associa o usuário a um tenant_id padrão.
- **POST /login:** Autenticação de usuários cadastrados.
- Retorna o accessToken (JWT) no corpo da resposta com tempo curto de expiração.
- Salva o refreshToken rotativo em um cookie de segurança HTTP-Only (passthrough: true, secure: true em prod, sameSite: strict, path: /api/v1/auth, maxAge: 30 dias).
- **POST /refresh:** Rotação automática de tokens (Refresh Token Rotation).
- Lê o refresh token do cookie, invalida o antigo, gera um novo par de tokens (Access + Refresh) e atualiza o cookie do cliente.
- **POST /logout:** Encerramento seguro da sessão.
- Limpa o cookie refreshToken no navegador/cliente e invalida a sessão no servidor.

Proteções e Autorizações

- **Rate Limiting:** Configurado globalmente via ThrottlerModule (limite padrão de 10 requisições por minuto por cliente).
 - **Guarda de Autenticação:** JwtAuthGuard implementado e ativado para proteger endpoints restritos da API.
 - **Controle de Acesso (RBAC):** RolesGuard e o decorator @Roles(...) criados para restringir endpoints de acordo com a regra de acesso do usuário (USER, ADMIN ou SUPER_ADMIN).
-

4. Observabilidade & Logs Estruturados

Em total conformidade com as diretrizes do projeto, foi implementado o serviço `StructuredLogger` no backend NestJS:

- **Logs Estruturados:** Cada evento crítico gera um log contendo metadados detalhados:
 - `timestamp`: Data e hora exata.
 - `level`: Severidade (Log, Warn, Error).
 - `service`: Nome do módulo/controlador emissor (ex: `auth-controller`).
 - `trace_id`: UUID único para rastreamento de requisições de ponta a ponta.
 - `user_id`: Identificação do usuário (quando logado).
 - `tenant_id`: Identificação do locatário/inquilino (multi-tenancy).
-

5. Pipelines de CI/CD e Deployment

A automação do projeto foi desenvolvida para suportar integração contínua (CI) e entrega contínua (CD) em produção:

- **Pipeline de CD (`cd-production.yml`):**

1. Executa validações automatizadas (`pnpm lint`, `pnpm typecheck`, `pnpm test`, `pnpm build`).
2. Executa migrações automáticas de banco de dados (`prisma migrate deploy`).
3. Deploy da API Backend no **Railway**.
4. Deploy do Painel Web Admin e Frontend no **Vercel**.

- **Deploy Local:** Setup pronto via `docker-compose.yml` para levantar a imagem do banco PostgreSQL com um clique.
-

6. Próximos Recursos Mapeados no Roadmap

Com a fundação técnica e a autenticação concluídas, os próximos domínios a serem implementados no monorepo são:

1. **API de Usuários e Onboarding** (chg-005): Armazenamento de preferências capturadas no onboarding.
2. **API de Lições e Trilhas** (chg-006): Distribuição de microlições e gerenciamento de conteúdo.
3. **API de Progresso & Streaks** (chg-007): Lógica de cálculo de ofensivas (estudos diários consecutivos).
4. **API de Gamificação** (chg-008): Liberação de insígnias, conquistas e ganho de pontos de experiência (XP).
5. **API de Revisão Inteligente** (chg-009): Implementação do algoritmo SuperMemo (SM-2) para revisões periódicas.
6. **API de Notificações** (chg-010): Lembretes e notificações push.
7. **Aplicativo Mobile (Fase 3)**: Design system com NativeWind e conexão com as APIs desenvolvidas.